What changes when a dependency becomes slow, unavailable, or uncertain?

Student notes for lecture 6. Campus Rooms leaves a single box: its parts run, wait for each other and fail separately. Each scene shows a situation, asks you to predict, reveals the mechanism, changes one condition, compares the outcomes and names the principle: three levels of architecture views, one request across real boundaries, a modular application against separate services, two racing bookings, a lost response and safe retries, a confirmed booking with a pending email, a contract and its consumers, and a decision record.

Scenes in this lecture

#	SCENE (STUDENT HEADING)	ROLE	DEEP LINK
01	What does a box labeled “Booking” stand for?	Bridge	`week-06.html#/architecture-zoom`
02	Follow one request across real boundaries	Anchor	`week-06.html#/request-journey`
03	What would separate services buy a small team?	Bridge	`week-06.html#/monolith-and-services`
04	Two sensible requests, one broken rule	Anchor	`week-06.html#/concurrent-booking`
05	A timeout does not tell you whether the write happened	Anchor	`week-06.html#/retry-and-idempotency`
06	A confirmed booking and a pending email can both be true	Bridge	`week-06.html#/async-and-consistency`
07	An added field and a changed meaning are not equivalent	Bridge	`week-06.html#/contracts-and-evolution`
08	Which structure protects the promises?	Bridge	`week-06.html#/architecture-decision`

Campus Rooms is a fictional room-booking service; every value is a teaching example, and every millisecond value is an illustrative label. The code is Python-like pseudocode.

Ari presses Book. No answer comes back. Is the room booked?

Ari (U-01) books room C101 from 10:00 to 11:00 in Campus Rooms. Watch what reaches the screen, then look at what happened inside.

What Ari sees

Ari (U-01) books C101 from 10:00 to 11:00. No answer: the request timed out.

Decide first

Is C101 booked for Ari? (A) No: an error means nothing was saved. (B) Yes: Campus Rooms always saves first. (C) Ari cannot tell from this screen.

What happened inside

- The request reached Booking.
- B-001 stored as Confirmed.
- The response was lost on the way back.

The booking database holds B-001: Ari (U-01), C101, 10:00 to 11:00, Confirmed. The write happened; the answer was lost. The timeout told Ari only that waiting stopped.

The question of this week

What changes when a dependency becomes slow, unavailable, or uncertain?

Five questions this lecture answers

- What does a box on an architecture diagram stand for? (scene 1)
- Which dependency does a request wait for, and which failures reach the user? (scenes 2–3)
- How can two sensible requests break one shared rule? (scene 4)
- What do you know after a timeout, and how do you retry safely? (scenes 5–6)
- How do contracts and decisions keep these promises over time? (scenes 7–8)

What you already know, and where this lecture goes

What changes when a dependency becomes slow, unavailable, or uncertain?

From Week 5

Carried forward: Modules with one responsibility each, contracts that include failure behavior, and a decision record with revisit triggers.

Campus Rooms: its components and two outside systems

- Booking: receives a booking request and answers it.
- Policy: owns the booking rules.
- Room Catalog: knows the rooms.
- Notification: sends confirmations.
- Storage: reads and writes the booking database.
- Identity: signs people in and vouches for who they are.
- Mail provider: delivers email.

Rules this lecture uses

- R-01: No overlapping confirmed bookings for the same room; 10:00 to 11:00 and 11:00 to 12:00 only touch.
- R-02: A booking lasts 30 to 120 minutes, both included.
- R-04: A failed notification never undoes a confirmed booking; pending email stays visible and recoverable.

By the end of this lecture you can

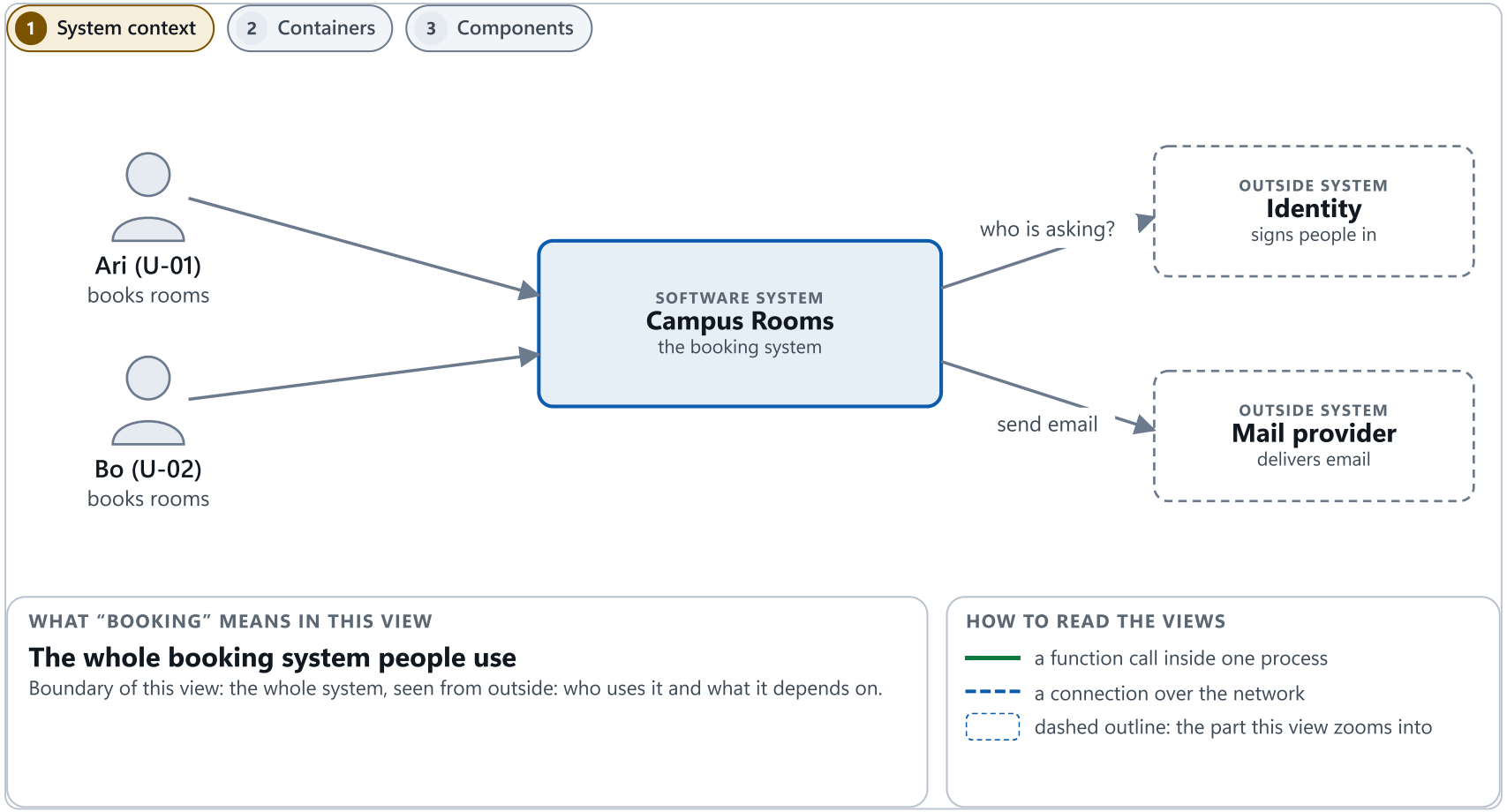
1. Distinguish a module boundary from a process or deployment boundary.
2. Read context, component, and runtime views at an appropriate level.
3. Explain how a concurrent check-then-write can violate a shared invariant.
4. Distinguish timeout, failure, and unknown outcome.
5. Compare synchronous and asynchronous interactions, including their costs.

What does a box labeled “Booking” stand for?

Panel 1 of 4 · Level 1 · system context: the whole system, seen from outside

Condition: **Baseline: one application, one database** State: **2 of 6**

Level 1 · system context. Booking is the whole booking system people use: who uses it and what it depends on.



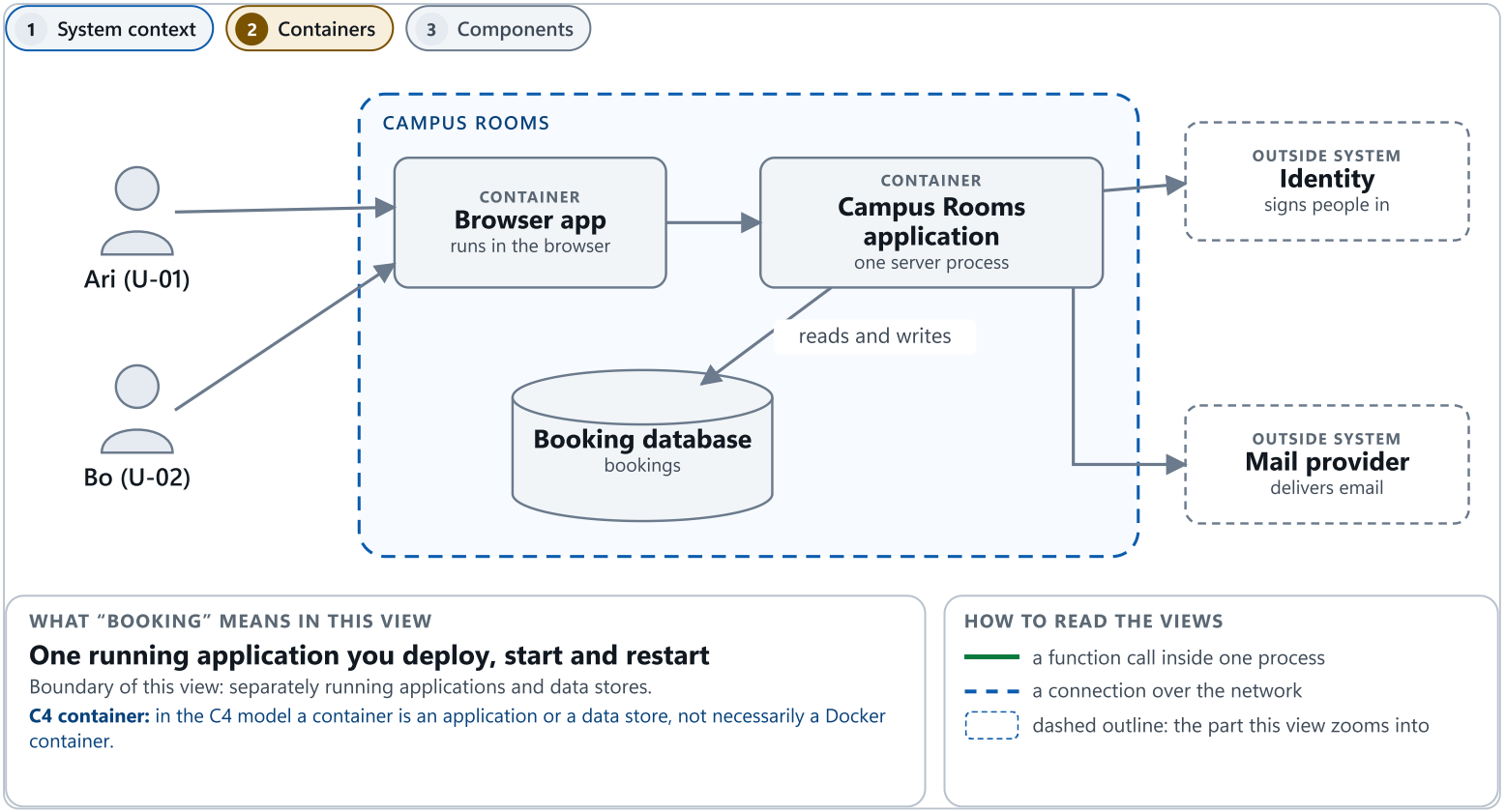
Booking here means the whole booking system people use. People: Ari (U-01), who books rooms and Bo (U-02), who books rooms. Outside systems: Identity (signs people in and vouches for who they are) and Mail provider (delivers email).

What does a box labeled “Booking” stand for?

Panel 2 of 4 · Level 2 · containers: what runs and what stores data

Condition: **Baseline: one application, one database** State: **3 of 6**

Level 2 · containers. Inside Campus Rooms: a browser app, one server application and one database. Booking is one running application you deploy, start and restart.



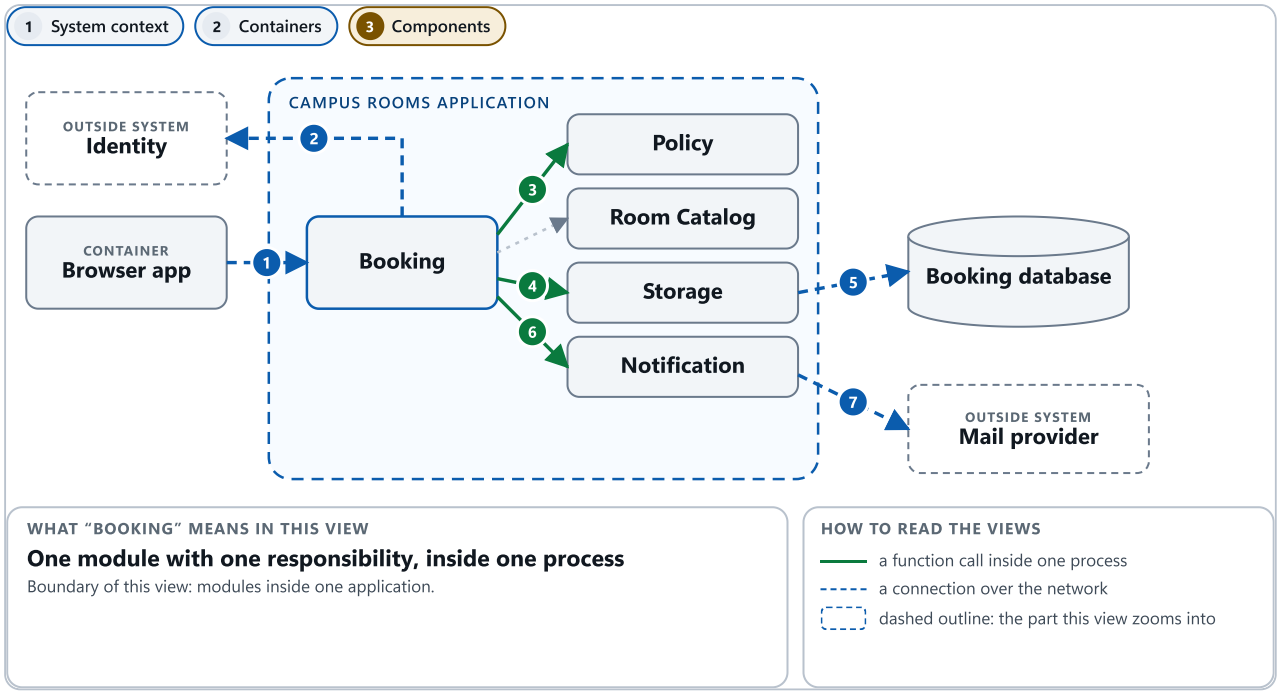
Booking here means one running application you deploy, start and restart. Browser app: the booking screens, runs in the browser; Campus Rooms application: decides and records bookings, one server process; Booking database: the authoritative record of bookings, database. In the C4 model a container is an application or a data store, not necessarily a Docker container.

What does a box labeled “Booking” stand for?

Panel 3 of 4 · Level 3 · components: one request traced inside the application

Condition: **Baseline: one application, one database** State: **6 of 6**

Five components, one process. Five logical parts are not five servers.



Booking here means one module with one responsibility, inside one process. The numbered hops of Ari's request: 1 Browser app to Booking, network; 2 Booking to Identity, network; 3 Booking to Policy, function call; 4 Booking to Storage, function call; 5 Storage to Booking database, network; 6 Booking to Notification, function call; 7 Notification to Mail provider, network. In short: three function calls inside one process; four network connections: the browser, Identity, the database and the mail provider. Solid arrows are function calls inside one process; dashed arrows cross the network.

Principle · Architecture views should state their level; logical separation does not automatically imply network separation.

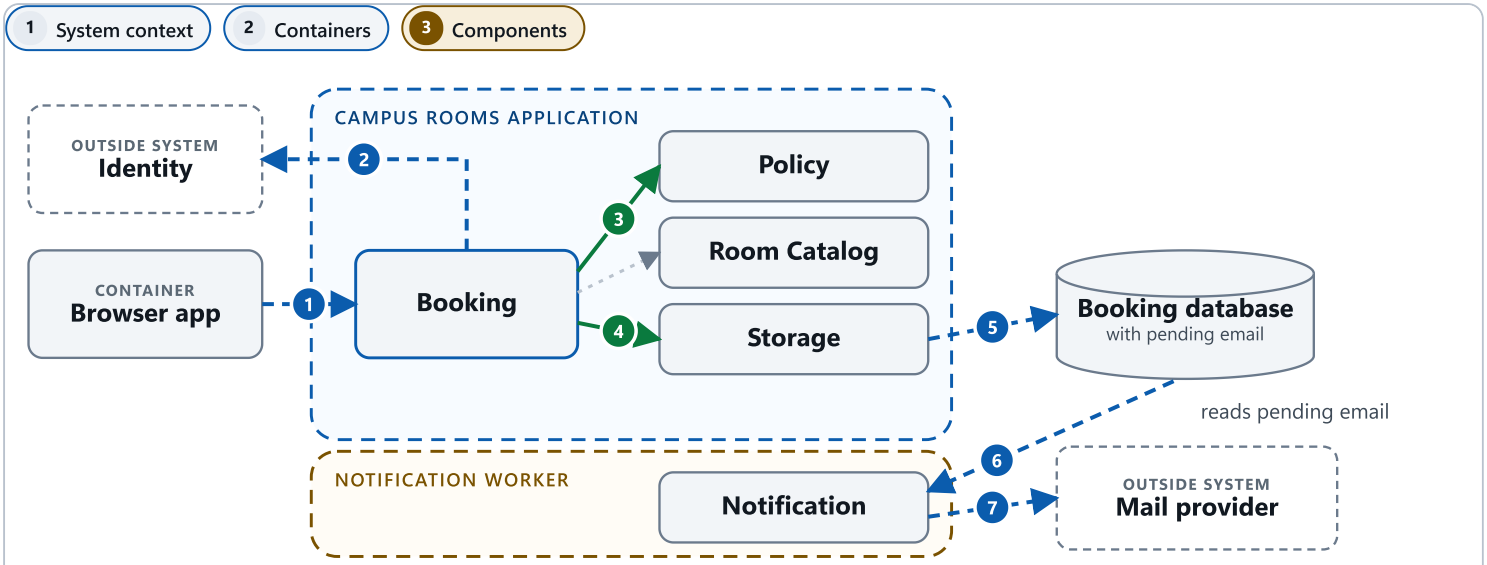
In short · Say which level a box lives on.

What does a box labeled “Booking” stand for?

Panel 4 of 4 · Changed: the notification worker as a second process

Condition: **Changed: Notification runs in its own worker** State: **4 of 5**

What changed is how it runs: one more process, one handoff, one new way to fail.



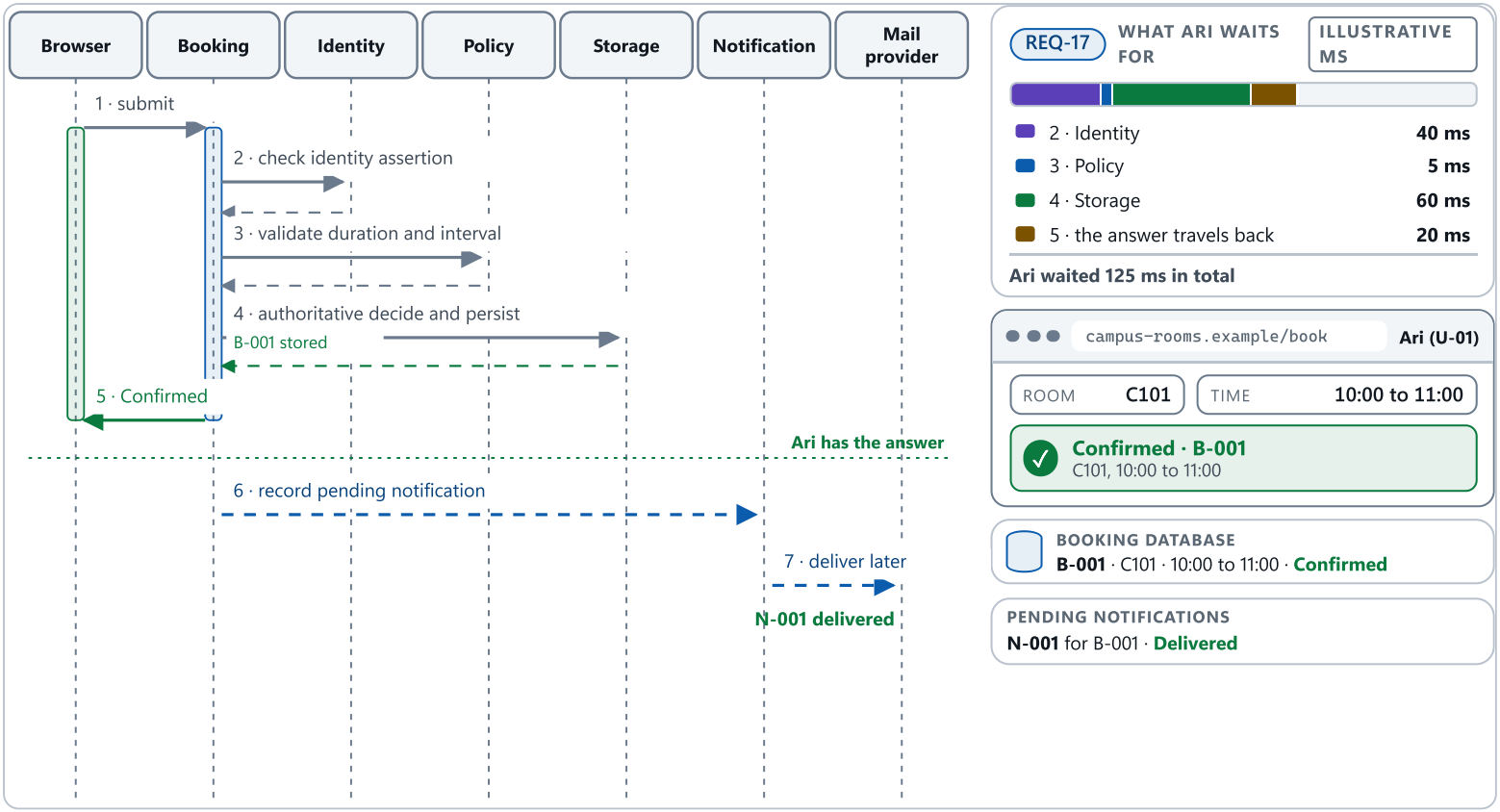
Notification moves into a separate worker process; Booking, Policy, Room Catalog and Storage stay in one application. Booking records a pending notification in the booking database; the worker reads it and delivers it. Changed in how it runs: two server processes to deploy, start and watch, instead of one; the worker restarts and scales without touching bookings; a handoff through the database replaces a function call; the worker can be down while bookings still confirm, so pending work must stay visible. The same in what it does: the same five responsibilities; the same rules, R-01 to R-04; Booking still decides; Policy still owns the rules; Ari still gets the same answer: Confirmed, B-001.

Follow one request across real boundaries

Panel 1 of 3 · A normal day: steps 1 to 5 hold up the answer, 6 and 7 follow it

Condition: **Baseline: every part answers** State: **9 of 9**

Steps 1 to 5 hold up the answer. Steps 6 and 7 continue after it.



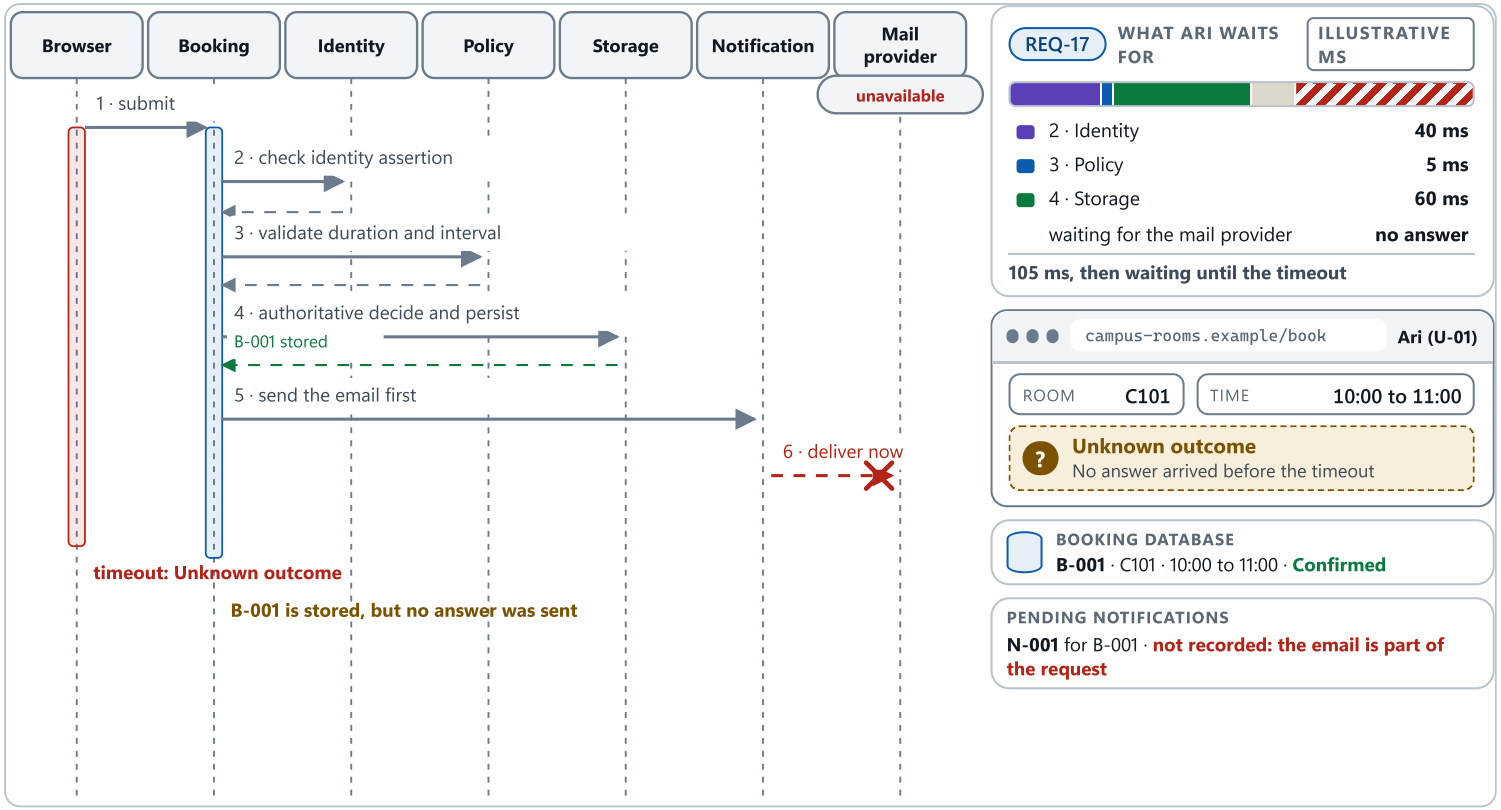
1. Browser to Booking: submit, Ari waits (0 ms); 2. Booking to Identity: check identity assertion, Ari waits (40 ms); 3. Booking to Policy: validate duration and interval, Ari waits (5 ms); 4. Booking to Storage: authoritative decide and persist, Ari waits (60 ms); 5. Booking to Browser: Confirmed, Ari waits (20 ms); 6. Booking to Notification: record pending notification, after the answer; 7. Notification to Mail provider: deliver later, after the answer. Millisecond values are illustrative labels, not measurements.

Follow one request across real boundaries

Panel 2 of 3 · The mail provider is down, and the email comes before the answer

Condition: **Changed: mail down, the email comes first** State: **5 of 5**

An email outage reached the booking answer: Ari cannot tell whether C101 is booked and may try again.



Response waits and times out; user sees Unknown outcome. Storage holds B-001 Confirmed; Ari cannot tell whether C101 is booked and may try again.

Principle · Interaction structure determines which failures affect which user promises.

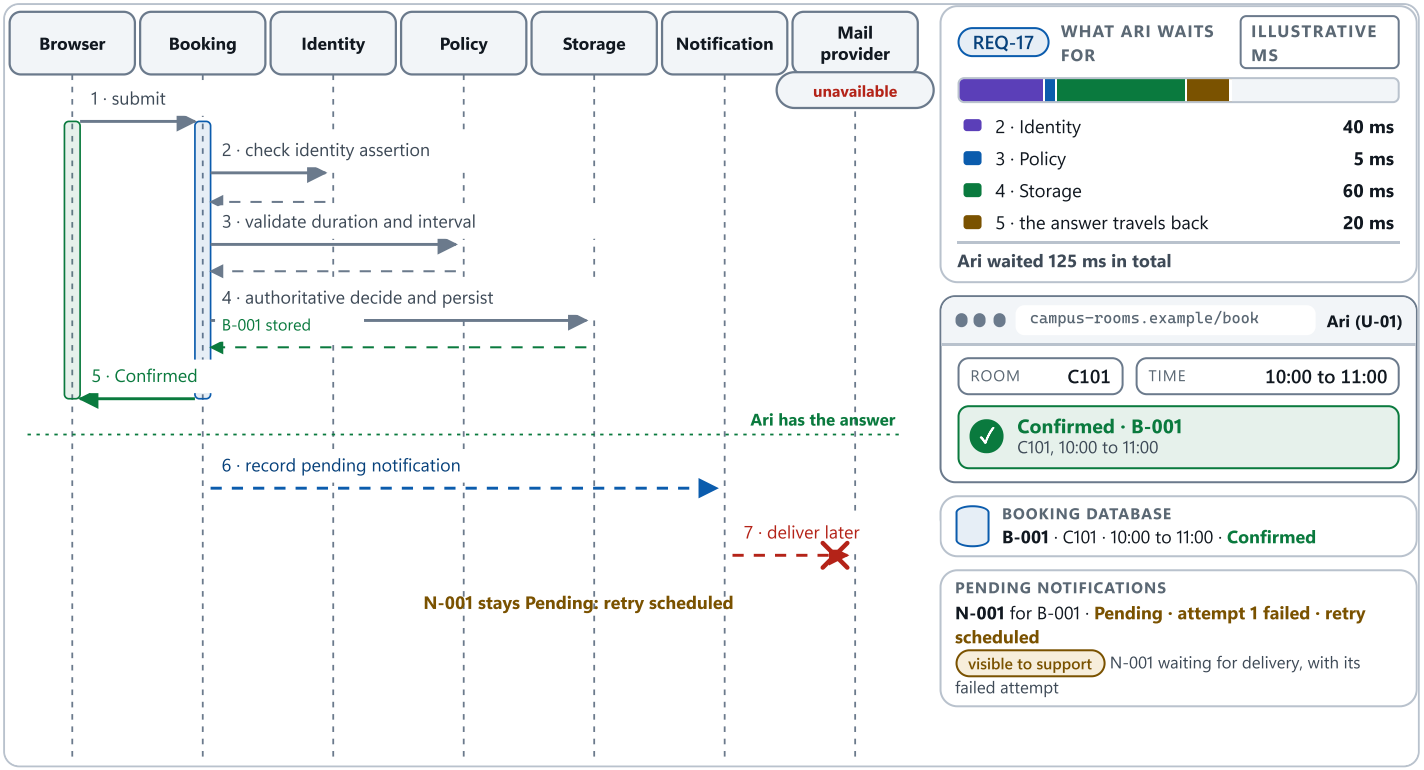
In short · What you wait for decides which failures reach you.

Follow one request across real boundaries

Panel 3 of 3 · The mail provider is down, and the email is pending work

Condition: **Changed: mail down, the email is pending work** State: **6 of 6**

Same failure, two designs: one breaks the booking promise, one only delays the email.



Confirmed at step 5; pending notification visible and recoverable (R-04). N-001 Pending; attempt 1 failed: mail provider unavailable; retry scheduled. Support sees N-001 waiting for delivery, with its failed attempt.

Principle · Interaction structure determines which failures affect which user promises.

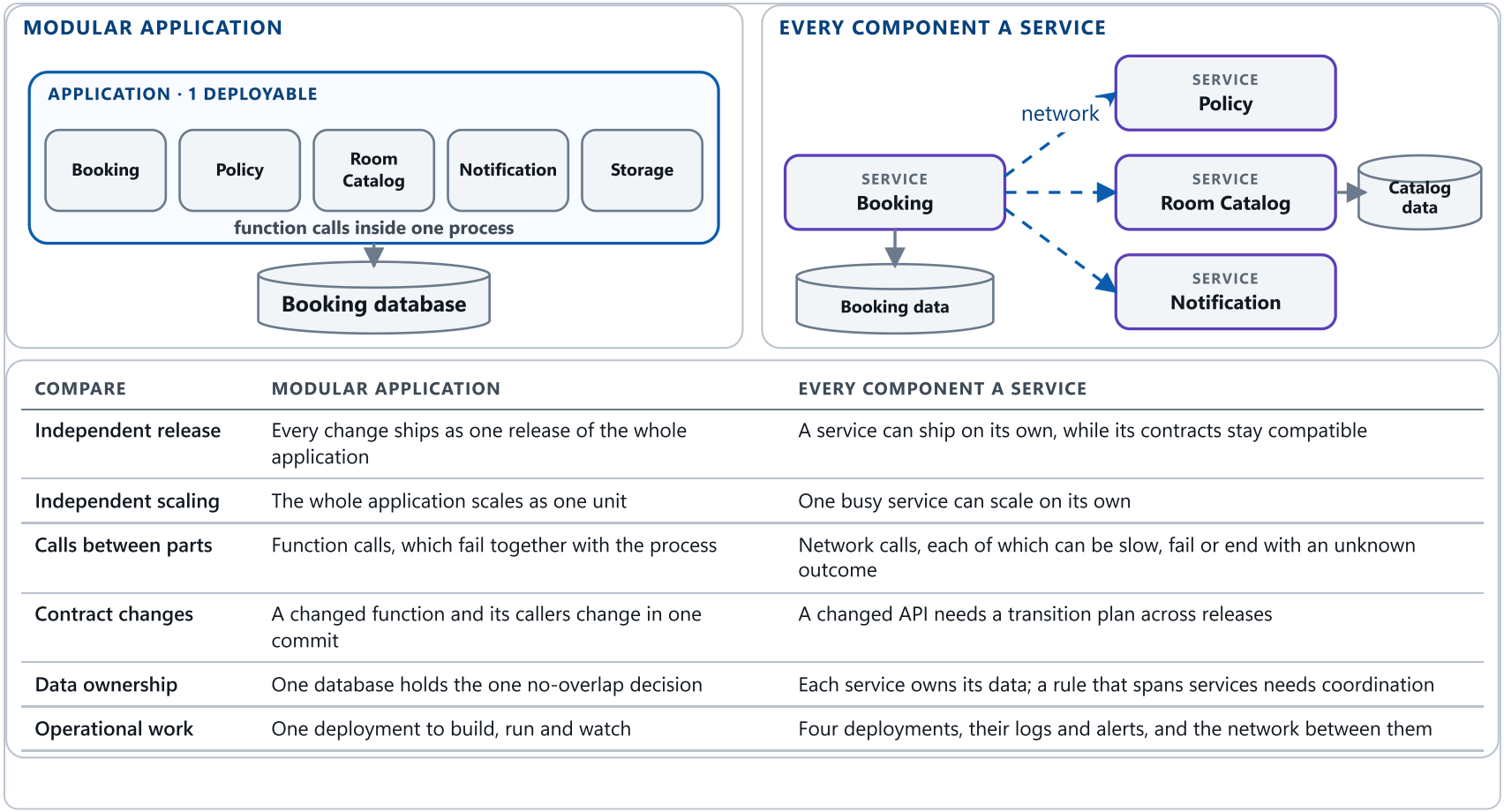
In short · What you wait for decides which failures reach you.

What would separate services buy a small team?

Panel 1 of 2 · One small team, steady load: both alternatives, no winner label

Condition: **Baseline: one small team, steady load** State: **8 of 8**

For one small team under steady load, the extra boundaries answer no need that has appeared.



COMPARE	MODULAR APPLICATION	EVERY COMPONENT A SERVICE
Independent release	Every change ships as one release of the whole application	A service can ship on its own, while its contracts stay compatible
Independent scaling	The whole application scales as one unit	One busy service can scale on its own
Calls between parts	Function calls, which fail together with the process	Network calls, each of which can be slow, fail or end with an unknown outcome
Contract changes	A changed function and its callers change in one commit	A changed API needs a transition plan across releases
Data ownership	One database holds the one no-overlap decision	Each service owns its data; a rule that spans services needs coordination
Operational work	One deployment to build, run and watch	Four deployments, their logs and alerts, and the network between them

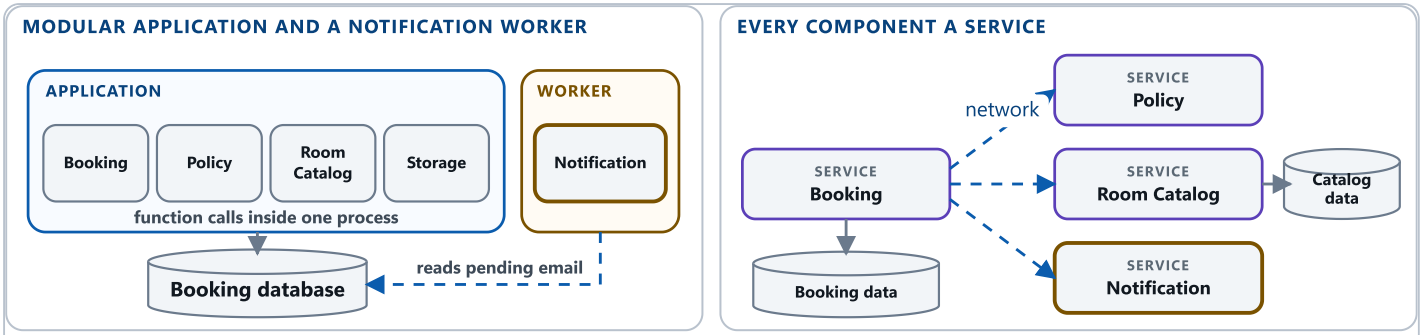
Modular application: 1 deployable (Booking, Policy, Room Catalog, Notification and Storage), one booking database. Every component a service: 4 deployables, Booking and Room Catalog each own their data. For one small team under steady load, the extra boundaries answer no need that has appeared.

What would separate services buy a small team?

Panel 2 of 2 · Changed: Notification volume grows independently of booking writes

Condition: **Changed: notifications grow, booking writes do not** State: **5 of 5**

Separating the one workload that grew targets the need; splitting every component adds boundaries the need did not ask for.



COMPARE	MODULAR APPLICATION AND A NOTIFICATION WORKER	EVERY COMPONENT A SERVICE
The growing workload	The Notification worker scales and fails on its own	The Notification service scales and fails on its own
Boundaries added	One handoff, through pending notifications in the database	Network calls between all four services, and split data
The booking path	Unchanged: one process decides and stores	Now crosses the network to Policy and Room Catalog

Modular application and a Notification worker: 2 deployables, one booking database, with pending notifications beside the bookings. Separating the one workload that grew targets the need; splitting every component adds boundaries the need did not ask for.

Principle · Choose deployment boundaries for demonstrated needs; modularity and distribution are different decisions.

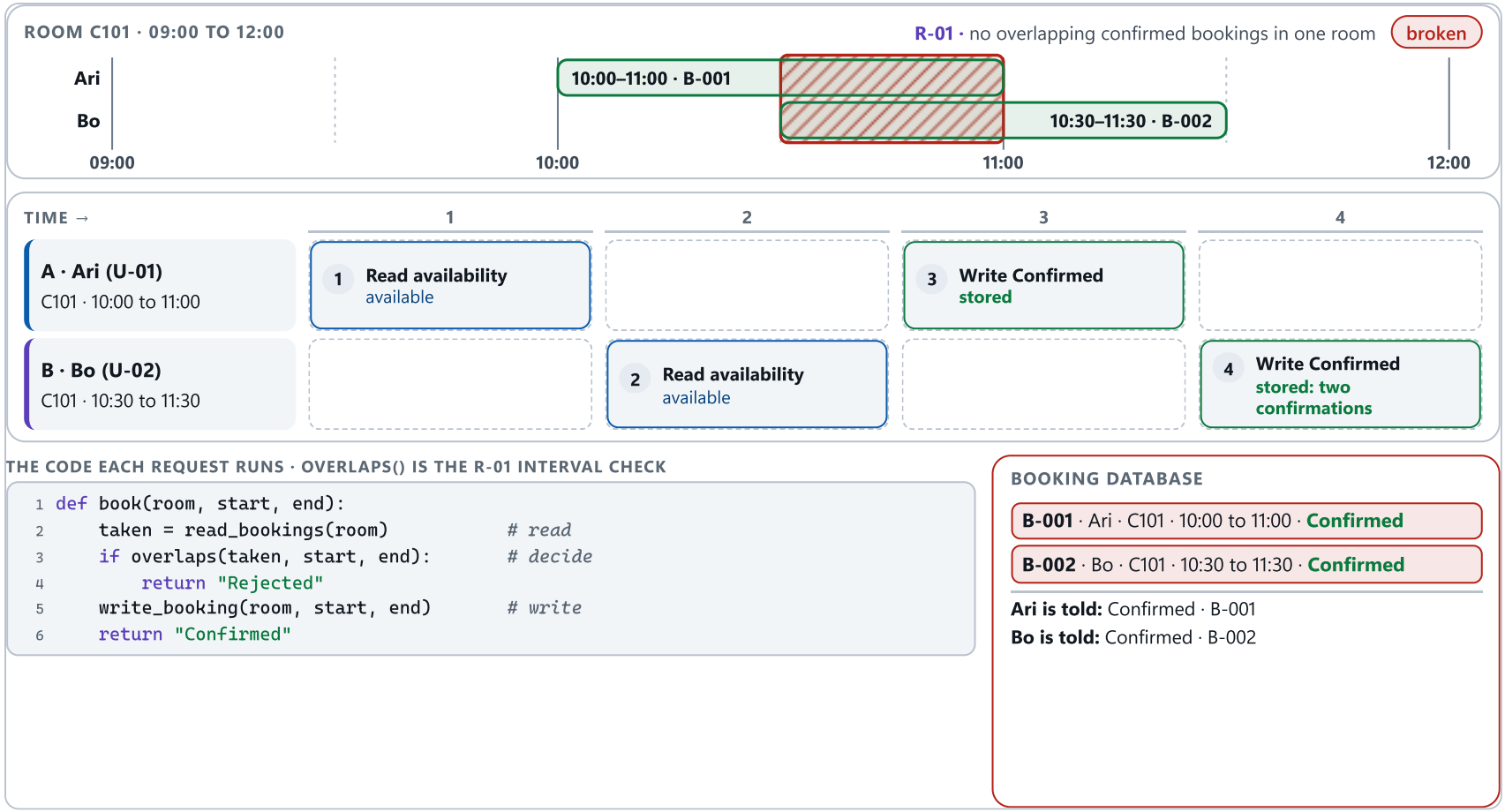
In short · Split a process only when a real need asks.

Two sensible requests, one broken rule

Panel 1 of 4 · Check, then write: both reads see the same earlier state

Condition: **Baseline: each request reads, decides, then writes** State: **6 of 6**

Each decision was sensible for what it read. The order of clicks does not matter: the rule broke between the read and the write.



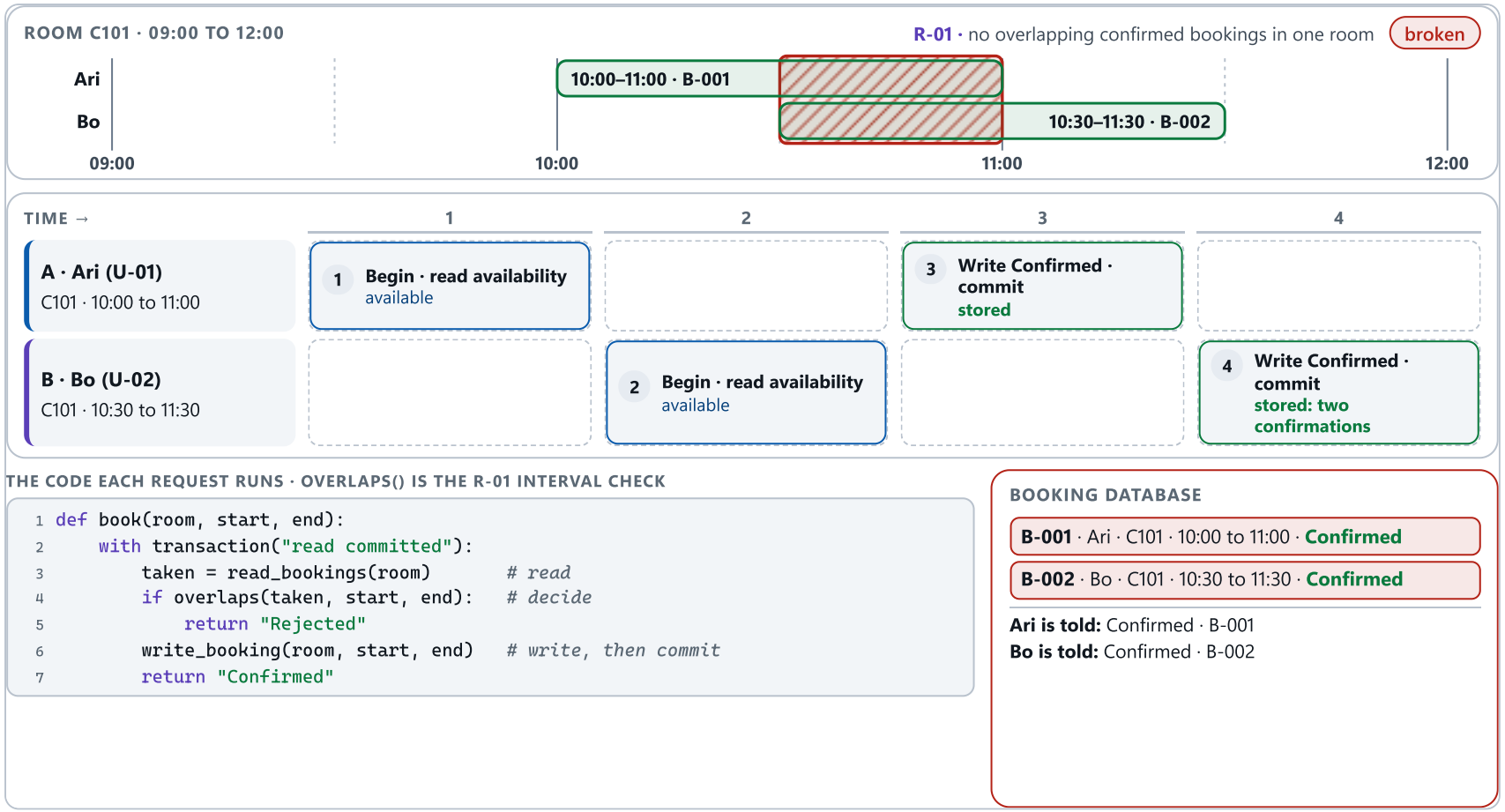
Order of events: 1. Ari's request: read availability, available; 2. Bo's request: read availability, available; 3. Ari's request: write Confirmed, stored; 4. Bo's request: write Confirmed, stored: two confirmations. R-01: no overlapping confirmed bookings for one room. 10:30 to 11:00 is booked twice.

Two sensible requests, one broken rule

Panel 2 of 4 · Inside a transaction at read committed

Condition: **Changed: inside a transaction, read committed** State: **6 of 6**

A transaction label is not a rule. Serializable isolation detects this pattern: one transaction fails and must be retried.



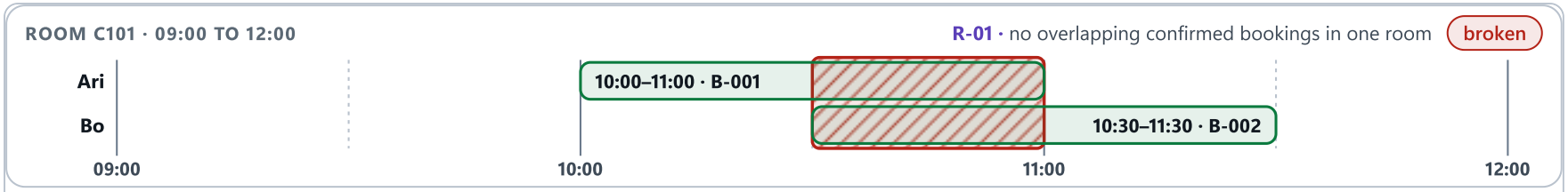
Two confirmations: each transaction reads before the other commits, and no conflicting row exists yet to lock. Serializable isolation detects this pattern: one transaction fails and must be retried.

Two sensible requests, one broken rule

Panel 3 of 4 · A unique key on room and start time

Condition: **Changed: a unique key on room and start** State: **6 of 6**

The key protects “same room, same start”, a smaller rule than R-01.



TIME →	1	2	3	4
A · Ari (U-01) C101 · 10:00 to 11:00	1 **Read availability** available		3 **Write key C101 · 10:00** accepted	
B · Bo (U-02) C101 · 10:30 to 11:30		2 **Read availability** available		4 **Write key C101 · 10:30** accepted: a different key

THE CODE EACH REQUEST RUNS · OVERLAPS() IS THE R-01 INTERVAL CHECK

```
# bookings: unique (room, start)
def book(room, start, end):
    taken = read_bookings(room)          # read
    if overlaps(taken, start, end):      # decide
        return "Rejected"
    write_booking(room, start, end)      # refused only for a repeated start
    return "Confirmed"
```

BOOKING DATABASE

- **B-001** · Ari · C101 · 10:00 to 11:00 · **Confirmed**
- **B-002** · Bo · C101 · 10:30 to 11:30 · **Confirmed**

Ari is told: Confirmed · B-001
Bo is told: Confirmed · B-002

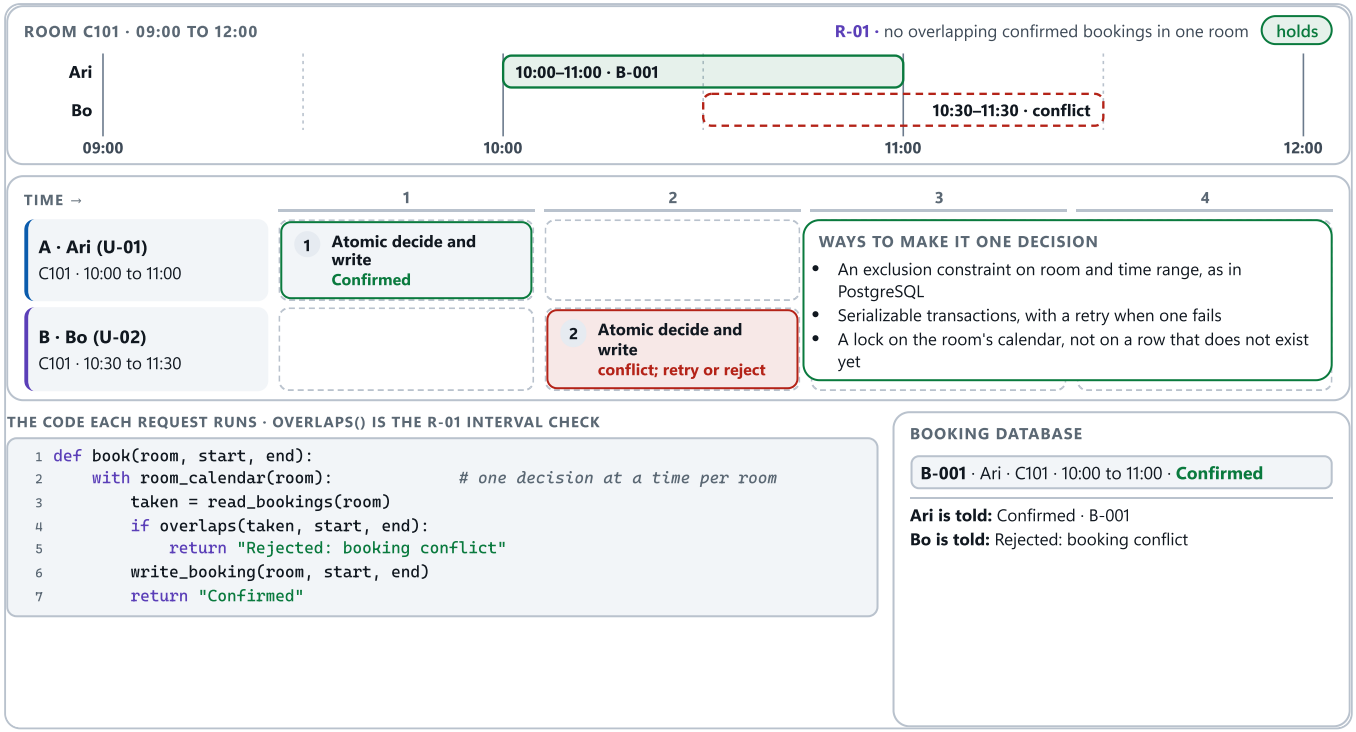
> Keys C101 · 10:00 and C101 · 10:30: two confirmations: the keys differ, the intervals still overlap.

Two sensible requests, one broken rule

Panel 4 of 4 · One atomic decision per room

Condition: **Changed: one atomic decision per room** State: **6 of 6**

Same two requests, same timing: now the rule survives.



1. Ari's request: atomic decide and write, Confirmed; 2. Bo's request: atomic decide and write, conflict; retry or reject. Bo sees: Rejected: C101 is booked from 10:00 to 11:00 (R-01). Ways to enforce it: an exclusion constraint on room and time range, as in PostgreSQL; serializable transactions, with a retry when one fails; a lock on the room's calendar, not on a row that does not exist yet. The strategy must enforce the interval invariant under interleavings; a transaction label alone does not.

Principle · An invariant shared by concurrent operations needs enforcement that survives their interleavings.

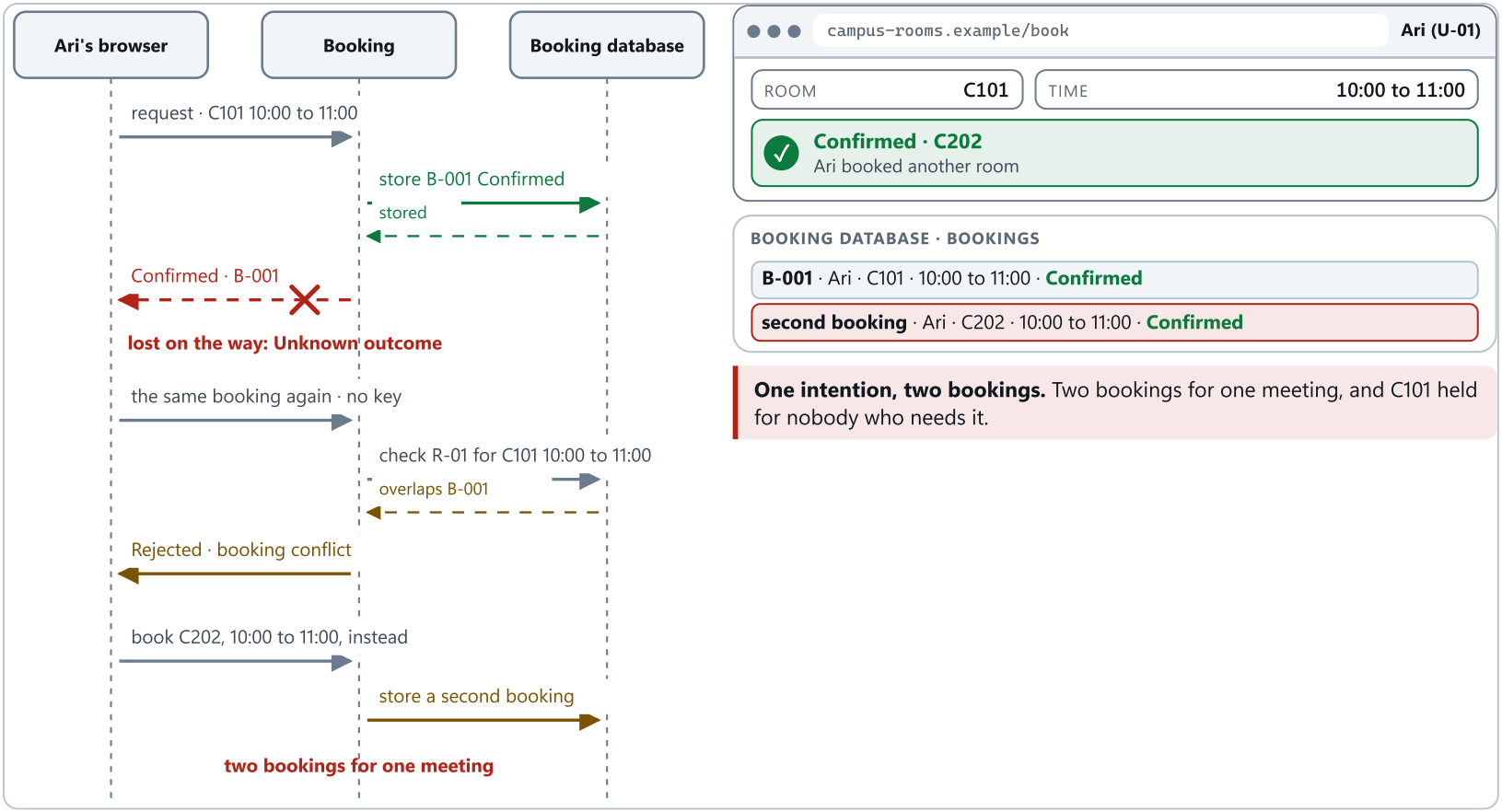
In short · A shared rule needs one guard that races cannot slip past.

A timeout does not tell you whether the write happened

Panel 1 of 3 · A lost response, then a retry with no key

Condition: **Baseline: the browser retries with no operation key** State: **7 of 7**

Two bookings for one meeting, and C101 held for nobody who needs it. Booking could not tell a retry from a new request.



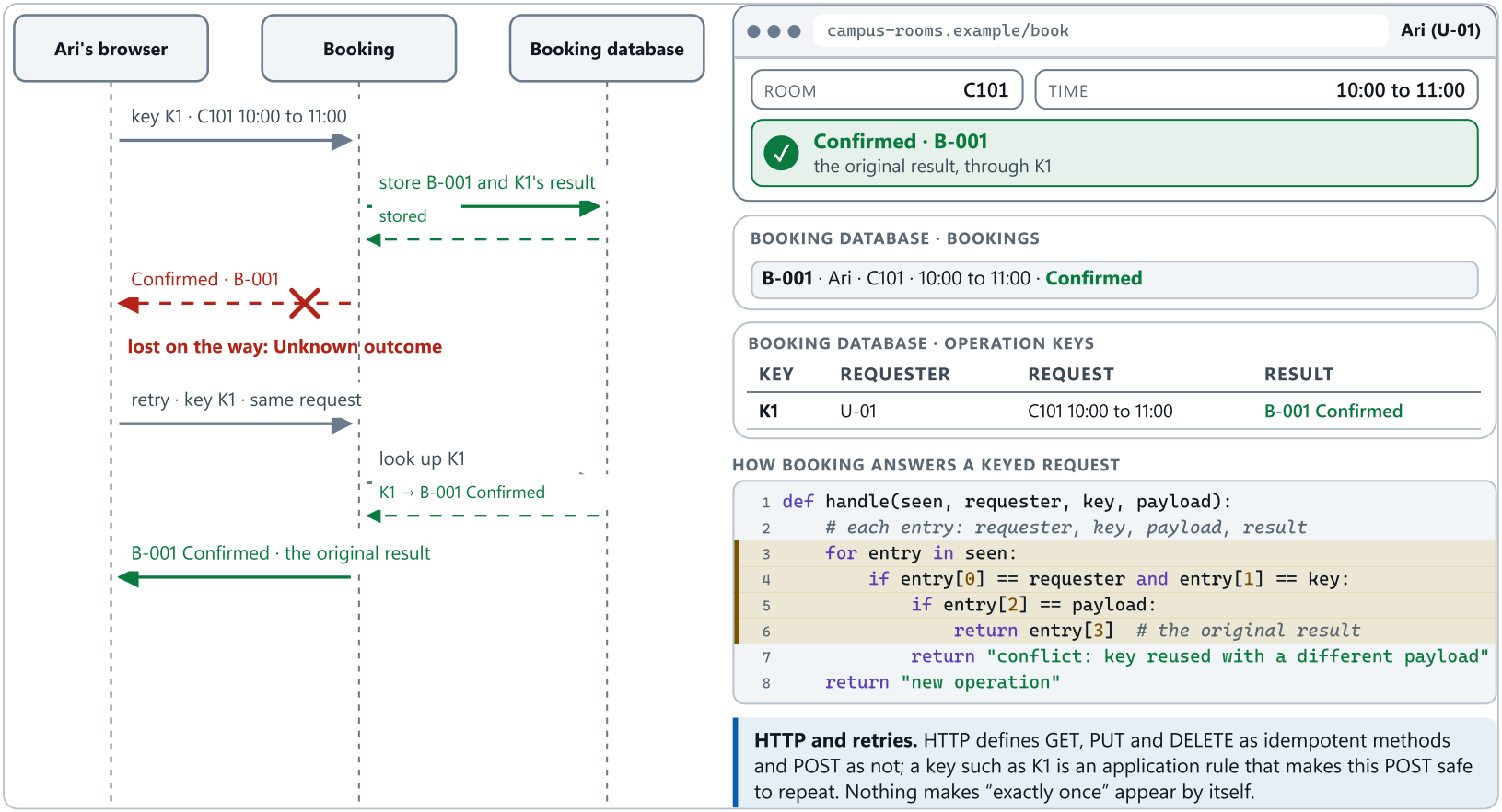
Ari's request carries only the booking, C101 10:00 to 11:00. Booking stores B-001 Confirmed; the response is lost, and the browser shows Unknown outcome. The retry carries no identity: Booking sees a new booking request for C101 from 10:00 to 11:00; Rejected: overlaps B-001 (R-01); Ari sees “Rejected: C101 is already booked”. Ari books C202 from 10:00 to 11:00 instead: two bookings for one meeting, and C101 held for nobody who needs it.

A timeout does not tell you whether the write happened

Panel 2 of 3 · The same key, K1, with the same request

Condition: **Changed: key K1, repeated on the retry** State: **5 of 5**

HTTP defines GET, PUT and DELETE as idempotent methods and POST as not; a key such as K1 is an application rule that makes this POST safe to repeat.



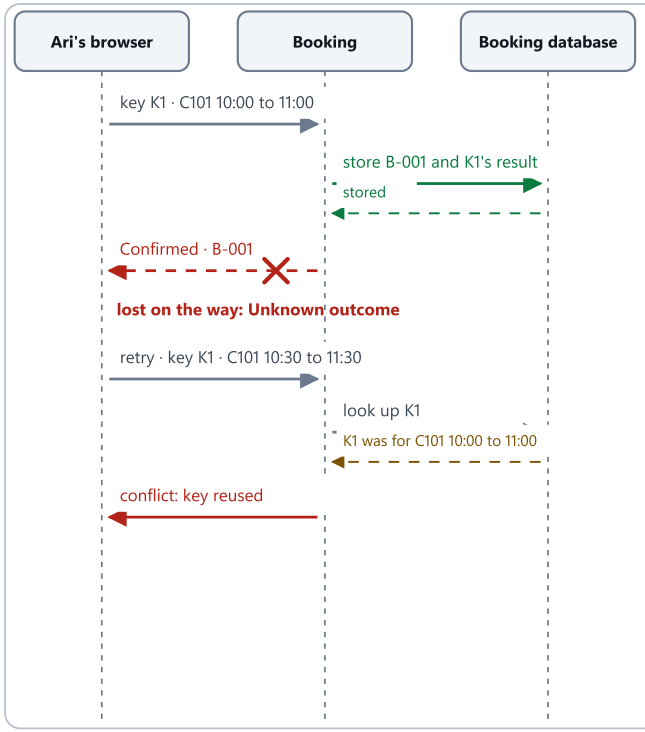
Key scope: requester U-01 and intended action: book C101 [10:00,11:00). Stored in the same commit as B-001. t3: retry with K1 and the same payload: original logical result: B-001 Confirmed; no second booking. HTTP defines GET, PUT and DELETE as idempotent methods and POST as not; a key such as K1 is an application rule that makes this POST safe to repeat.

A timeout does not tell you whether the write happened

Panel 3 of 3 · K1 reused with a different request

Condition: **Changed: K1 reused for 10:30 to 11:30** State: **4 of 4**

One identity per intended action: repeat it, and the result repeats; change the action, and the key refuses.



campus-rooms.example/book — Ari (U-01)

ROOM C101 TIME 10:00 to 11:00

Conflict: key reused with a different request
nothing changed

BOOKING DATABASE · OPERATION KEYS

KEY	REQUESTER	REQUEST	RESULT
K1	U-01	C101 10:00 to 11:00	B-001 Confirmed

THE SAME HANDLER, FOUR REQUESTS (CHECKED RESULTS)

STORED KEY	FROM · KEY	REQUEST	ANSWER
K1 → B-001 Confirmed	U-01 · K1	C101 10:00 to 11:00	B-001 Confirmed
K1 → B-001 Confirmed	U-01 · K1	C101 10:30 to 11:30	conflict: key reused with a different payload
K1 → B-001 Confirmed	U-02 · K1	C101 10:00 to 11:00	**new operation**
nothing yet	U-01 · K1	C101 10:00 to 11:00	**new operation**

t4: retry with K1 and a different payload (10:30-11:30): explicit conflict: key reused with a different payload. The handler's checked answers: U-01 with K1 and C101 10:00 to 11:00: B-001 Confirmed; U-01 with K1 and C101 10:30 to 11:30: conflict: key reused with a different payload; U-02 with K1 and C101 10:00 to 11:00: new operation; U-01 with K1 and C101 10:00 to 11:00 (nothing stored yet): new operation.

Principle · Retries need defined semantics for identity, repeated effects, and uncertain outcomes.

In short · Give each operation an identity, so a retry cannot double it.

Four answers Ari can get. What does each one say about the write?

Ari asked for C101 10:00 to 11:00. Some answers come from Booking, one only from Ari's own clock. Decide for each before you open it.

Ari's request: C101 10:00 to 11:00

- Rejected: booking conflict (a response from Booking). Stored? No booking stored: Booking decided, and the answer is no. Next: choose another time; the same request gets the same answer.
- Service unavailable (a response from Booking). Stored? No booking stored: the contract sends this only before any decision. Next: retry later with the same key.
- Unexpected server error (a response, but not a decision). Stored? Unknown: the failure may have come after the write. Next: treat it as an unknown outcome: ask or retry with the same key.
- No answer before the timeout (Ari's own clock, not Booking). Stored? Unknown: the request may be lost, still running, or stored with its answer lost. Next: ask or retry with the same key; never assume nothing happened.

Three observations, three names

- Timeout: the waiting side stopped waiting; an observation at one boundary.
- Failure: a response that reports a problem; it says only what its contract promises.
- Unknown outcome: the requester cannot tell whether the write happened.

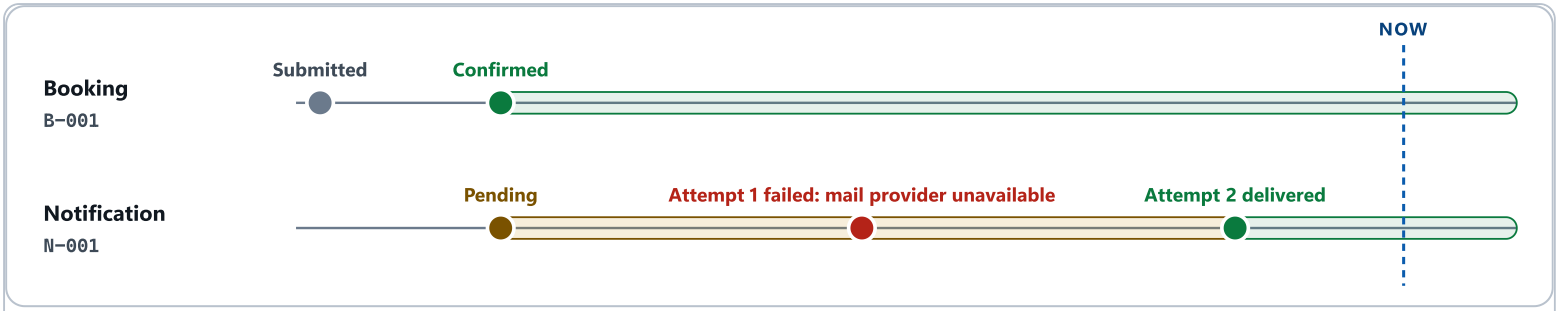
An error message is not always a no. Two of the four answers leave the write unknown; the same operation key turns them into a safe question.

A confirmed booking and a pending email can both be true

Panel 1 of 3 · B-001 and N-001: two timelines and what each combination means

Condition: **Baseline: B-001 and N-001 on two timelines** State: **5 of 6**

What each combination means to the person who booked, and to support.



ARI SEES

Confirmed, and the email arrived

Inbox: "Your booking B-001 is confirmed: C101, 10:00 to 11:00"

SUPPORT SEES

Delivered N-001 delivered

BOOKING	EMAIL	ARI IS TOLD	SUPPORT SEES
Confirmed	Pending	Your booking is confirmed; the email is on its way	N-001 waiting for delivery
Confirmed	Failed delivery	Your booking is confirmed; the email is late	N-001 failed once, retry scheduled
Confirmed	Delivered	Confirmed, and the email arrived	N-001 delivered
Rejected	none	No booking, so no confirmation email	No notification record

B-001: Submitted → Confirmed. N-001: Pending → Attempt 1 failed: mail provider unavailable → Attempt 2 delivered. Confirmed with pending: "Your booking is confirmed; the email is on its way"; support sees N-001 waiting for delivery. Confirmed with failed delivery: "Your booking is confirmed; the email is late"; support sees N-001 failed once, retry scheduled. Confirmed with delivered: "Confirmed, and the email arrived"; support sees N-001 delivered. Rejected with no email: "No booking, so no confirmation email"; support sees no notification record.

A confirmed booking and a pending email can both be true

Panel 2 of 3 · A duplicate delivery attempt of N-001

Condition: **Changed: N-001 is attempted twice** State: **4 of 4**

B-001 stays Confirmed in both. Only the handler decides whether repeat work is harmless.

B-001 · C101 · 10:00 to 11:00 · **Confirmed** · unchanged by any attempt

HANDLER THAT CHECKS FIRST

Attempt 1	send N-001	delivered
Attempt 2	finds N-001 already delivered and does nothing	nothing new

Ari's inbox **1 email**

One email

HANDLER THAT ALWAYS SENDS

Attempt 1	send N-001	delivered
Attempt 2	sends N-001 again	sent again

Ari's inbox **2 emails**

A second email to Ari and two sends in the log

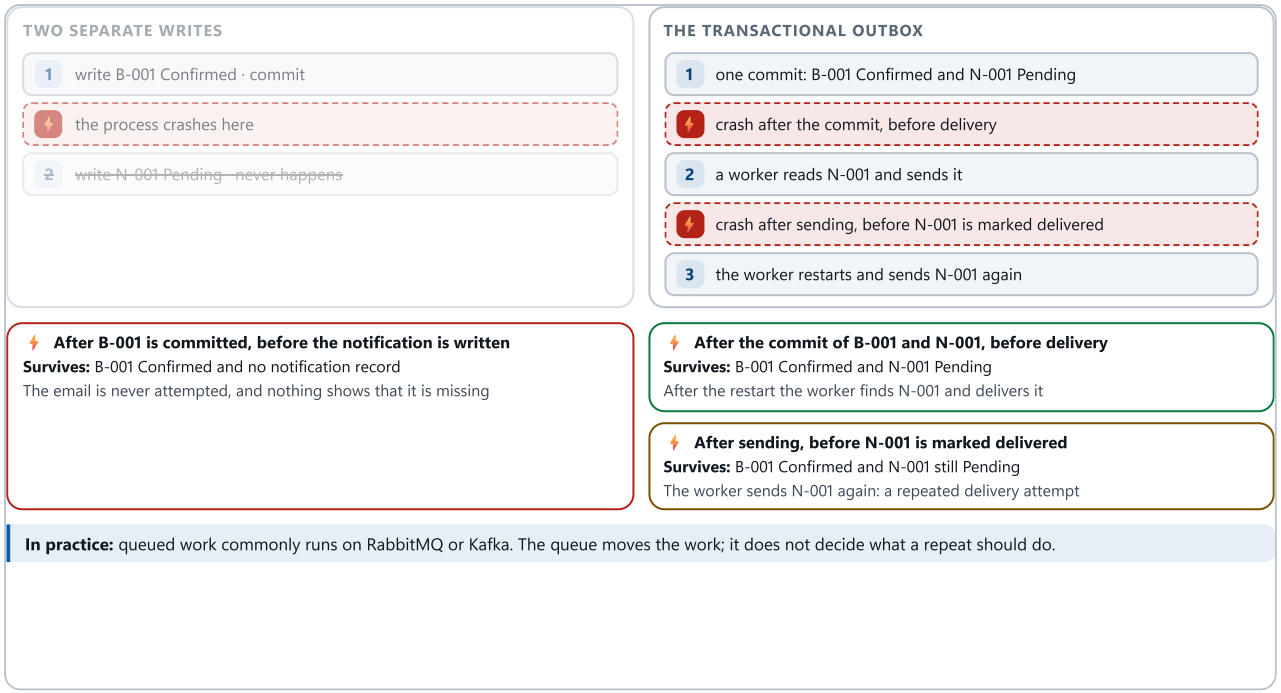
Handler that checks first: finds N-001 already delivered and does nothing, one email. Handler that always sends: sends N-001 again, a second email to Ari and two sends in the log. B-001 stays Confirmed in both.

A confirmed booking and a pending email can both be true

Panel 3 of 3 · Crash points with two separate writes and with a transactional outbox

Condition: **Changed: crashes at three moments** State: **6 of 6**

The outbox keeps the work discoverable. The handler still has to tolerate a repeat.



Two separate writes, after B-001 is committed, before the notification is written: B-001 Confirmed and no notification record; the email is never attempted, and nothing shows that it is missing. Transactional outbox, after the commit of B-001 and N-001, before delivery: B-001 Confirmed and N-001 Pending; after the restart the worker finds N-001 and delivers it. Transactional outbox, after sending, before N-001 is marked delivered: B-001 Confirmed and N-001 still Pending; the worker sends N-001 again: a repeated delivery attempt. No exactly-once email guarantee.

Principle · Asynchronous work changes the timing and failure contract; it does not remove responsibility for completion.

In short · Later work is still your work: track it until done.

An added field and a changed meaning are not equivalent

Panel 1 of 4 · The booking contract, POST /bookings

Condition: **Baseline: the booking contract today** State: **8 of 8**

Three consumers read today's answers, each in its own way. All three work.

POST /bookings Booking contract

REQUEST `room_id` C101 · `start` 10:00 · `end` 11:00 · `operation_key` K1

IDENTITY The signed-in requester, from Identity; never a field the browser fills in

RESPONSE `booking_id` B-001 · `outcome` Confirmed

ERRORS

- **invalid request** · a field is missing or the times are malformed
- **not permitted** · the requester may not do this
- **booking conflict** · R-01 or R-02 says no
- **service unavailable** · sent only before any decision

RETRY RULE Repeat the same operation_key with the same request to get the original result; the same key with a different request is a conflict.

An OpenAPI description captures
The operation, its fields, their types and the error responses

It does not establish

- who owns the booking
- that bookings never overlap
- that a retry is safe

RESPONSE THAT BOOKING SENDS

```
{
  "booking_id": "B-001",
  "outcome": "Confirmed"
}
```

Booking web app
Reads booking_id and outcome
Ignores fields it does not know
works
reads today's answers

Room door display
Reads outcome
Ignores fields it does not know; shows a room as booked when outcome is "Confirmed"
works
reads today's answers

Nightly export
Reads every field
Rejects any field or value it does not know
works
reads today's answers

Request: room_id (C101), start (10:00), end (11:00), operation_key (K1). Identity: the signed-in requester, from Identity; never a field the browser fills in. Response: booking_id (B-001), outcome (Confirmed). Errors: invalid request, a field is missing or the times are malformed; not permitted, the requester may not do this; booking conflict, R-01 or R-02 says no; service unavailable, sent only before any decision. Repeat the same operation_key with the same request to get the original result; the same key with a different request is a conflict. An OpenAPI description captures the operation, its fields, their types and the error responses; it does not establish who owns the booking, that bookings never overlap and that a retry is safe.

An added field and a changed meaning are not equivalent

Panel 2 of 4 · Added field: room_name

Condition: **Changed: add a response field: room_name** State: **6 of 6**

Adding a field is safe only for consumers that ignore what they do not know.

POST /bookings Booking contract

REQUEST	`room_id` C101 · `start` 10:00 · `end` 11:00 · `operation_key` K1
IDENTITY	The signed-in requester, from Identity; never a field the browser fills in
RESPONSE	`booking_id` B-001 · `outcome` Confirmed
ERRORS	• **invalid request** · a field is missing or the times are malformed • **not permitted** · the requester may not do this • **booking conflict** · R-01 or R-02 says no • **service unavailable** · sent only before any decision
RETRY RULE	Repeat the same operation_key with the same request to get the original result; the same key with a different request is a conflict.

RESPONSE THAT BOOKING SENDS

```
{
  "booking_id": "B-001",
  "outcome": "Confirmed",
  "room_name": "Room C101"
}
```

Booking web app
Reads booking_id and outcome
Ignores fields it does not know
works
shows the room name

Room door display
Reads outcome
Ignores fields it does not know; shows a room as booked when outcome is "Confirmed"
works
ignores room_name

Nightly export
Reads every field
Rejects any field or value it does not know
fails
rejects the unknown field room_name

Booking web app (ignores fields it does not know): works, shows the room name. Room door display (ignores fields it does not know; shows a room as booked when outcome is "Confirmed"): works, ignores room_name. Nightly export (rejects any field or value it does not know): fails, rejects the unknown field room_name.

An added field and a changed meaning are not equivalent

Panel 3 of 4 · Same field, new meaning

Condition: **Changed: same field, new meaning** State: **4 of 4**

A changed meaning passes every shape check, an OpenAPI description included.

POST /bookings Booking contract

REQUEST	`room_id` C101 · `start` 10:00 · `end` 11:00 · `operation_key` K1
IDENTITY	The signed-in requester, from Identity; never a field the browser fills in
RESPONSE	`booking_id` B-001 · `outcome` Confirmed
ERRORS	• **invalid request** · a field is missing or the times are malformed • **not permitted** · the requester may not do this • **booking conflict** · R-01 or R-02 says no • **service unavailable** · sent only before any decision
RETRY RULE	Repeat the same operation_key with the same request to get the original result; the same key with a different request is a conflict.

An OpenAPI description captures
The operation, its fields, their types and the error responses

It does not establish
- who owns the booking
- that bookings never overlap
- that a retry is safe

RESPONSE THAT BOOKING SENDS

```
{
  "booking_id": "B-001",
  "outcome": "Confirmed"
}
```

Same bytes; now sent before R-01 is checked

Booking web app
Reads booking_id and outcome
Ignores fields it does not know
silently wrong
tells Bo the overlapping request is booked

Room door display
Reads outcome
Ignores fields it does not know; shows a room as booked when outcome is "Confirmed"
silently wrong
shows a booking that R-01 later rejects

Nightly export
Reads every field
Rejects any field or value it does not know
silently wrong
exports rejected requests as bookings

Same field, new meaning: outcome "Confirmed" is sent as soon as a request arrives, before R-01 is checked. Booking web app: silently wrong, tells Bo the overlapping request is booked. Room door display: silently wrong, shows a booking that R-01 later rejects. Nightly export: silently wrong, exports rejected requests as bookings.

An added field and a changed meaning are not equivalent

Panel 4 of 4 · Producer and consumer compatibility matrix

Condition: **Changed: new status values** State: **7 of 7**

The whole picture: each change, each consumer, and the assumption behind each cell.

`POST /bookings` Booking contract

REQUEST	`room_id` C101 · `start` 10:00 · `end` 11:00 · `operation_key` K1
IDENTITY	The signed-in requester, from Identity; never a field the browser fills in
RESPONSE	`booking_id` B-001 · `outcome` Confirmed
ERRORS	• **invalid request** · a field is missing or the times are malformed • **not permitted** · the requester may not do this • **booking conflict** · R-01 or R-02 says no • **service unavailable** · sent only before any decision
RETRY RULE	Repeat the same operation_key with the same request to get the original result; the same key with a different request is a conflict.

RESPONSE THAT BOOKING SENDS

```
{
  "booking_id": "B-001",
  "status": "CONFIRMED"
}
```

CHANGE	BOOKING WEB APP ignores fields it does not know	ROOM DOOR DISPLAY ignores fields it does not know	NIGHTLY EXPORT rejects any field or value it does not know
Add room_name	works shows the room name	works ignores room_name	fails rejects the unknown field room_name
Same field, new meaning	silently wrong tells Bo the overlapping request is booked	silently wrong shows a booking that R-01 later rejects	silently wrong exports rejected requests as bookings
"Confirmed" to "CONFIRMED" in one release	works released together with the producer	silently wrong shows C101 as free	fails rejects the unknown value
A transition through status	works moves to status	works keeps reading outcome until it moves	works accepts status before it appears

Transition steps: the export learns the new field and value first; the producer sends status beside outcome; each consumer moves to status in its own release; outcome is removed once no consumer reads it. Each cell's assumption is the consumer's reading style: Booking web app, ignores fields it does not know; Room door display, ignores fields it does not know; shows a room as booked when outcome is "Confirmed"; Nightly export, rejects any field or value it does not know.

Principle · Compatibility is a relationship between actual producers and consumers, not a label on a release.

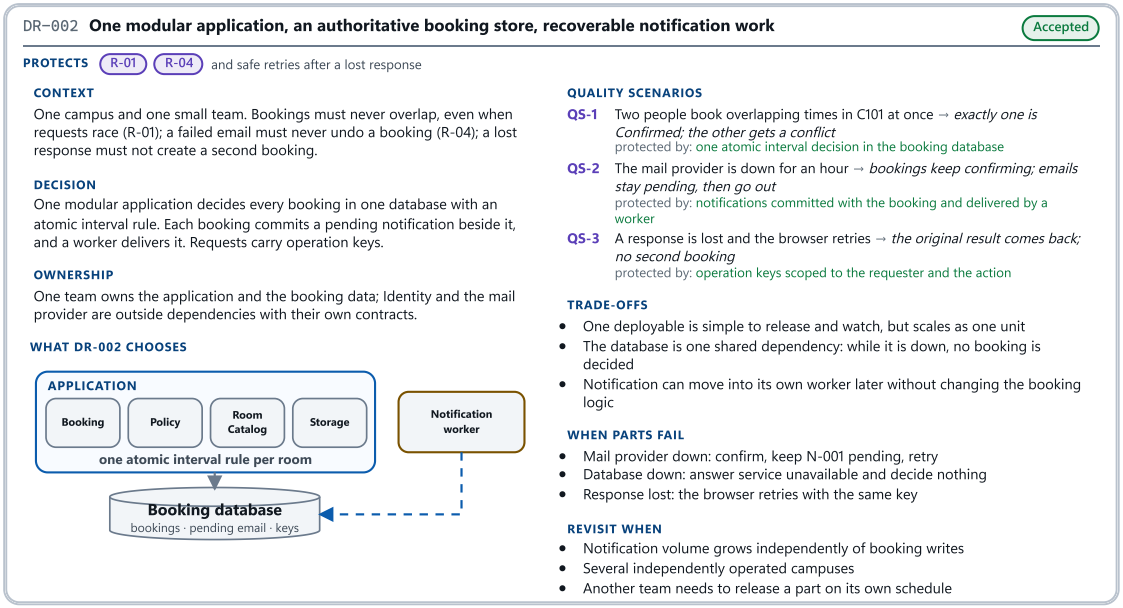
In short · Ask who reads a field before you change it.

Which structure protects the promises?

Panel 1 of 2 · DR-002: One modular application, an authoritative booking store, recoverable notification work

Condition: **Baseline: one campus and one small team** State: **7 of 7**

Revisit when names the changes that would reopen the decision. A teaching baseline for this case, not a prescription for every system.



Status: Accepted. Protects R-01 and R-04 and safe retries. Context: One campus and one small team. Bookings must never overlap, even when requests race (R-01); a failed email must never undo a booking (R-04); a lost response must not create a second booking. Decision: One modular application decides every booking in one database with an atomic interval rule. Each booking commits a pending notification beside it, and a worker delivers it. Requests carry operation keys. Revisit when: Notification volume grows independently of booking writes; several independently operated campuses; another team needs to release a part on its own schedule. A teaching baseline for this case, not a prescription for every system.

Principle · Architecture decisions should explain which requirements they protect and which assumptions would change the choice.

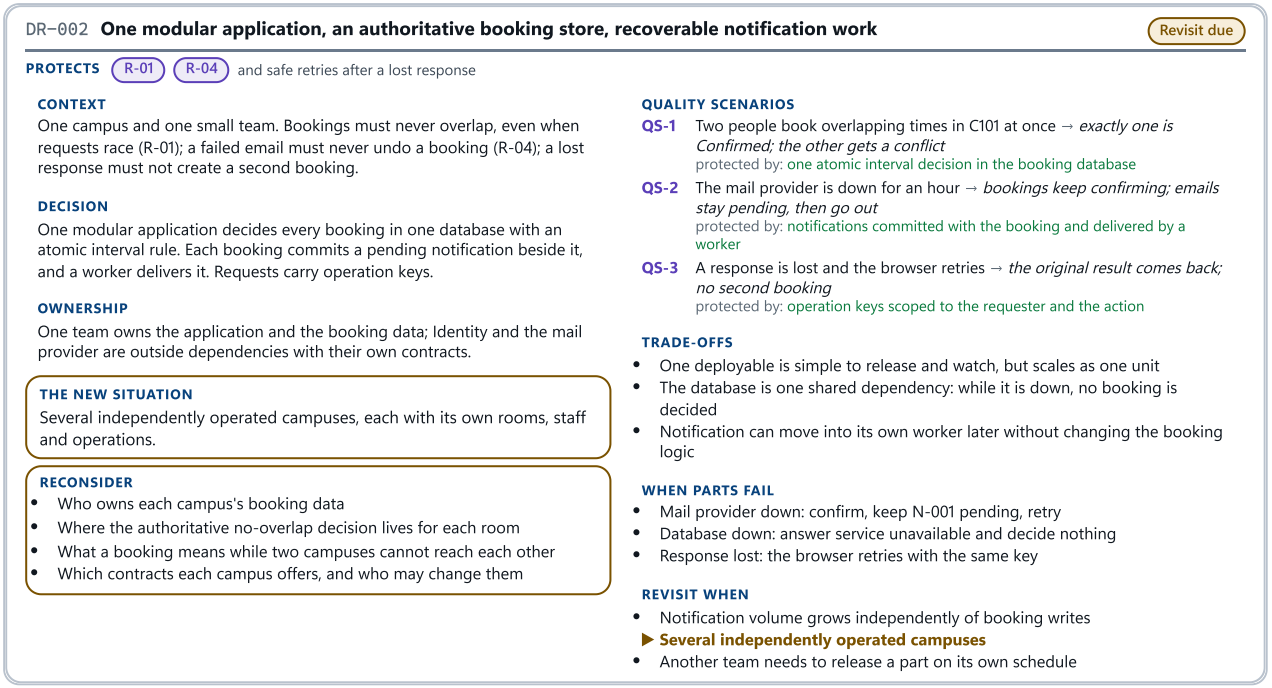
In short · Write down what a choice protects and what would change it.

Which structure protects the promises?

Panel 2 of 2 · Changed: several independently operated campuses, each with its own rooms, staff and operations

Condition: **Changed: several independently operated campuses** State: **4 of 4**

Consistency across distant sites is not solved in this lecture; the record names the questions and keeps the decision open.



The trigger “Several independently operated campuses” fires. Reconsider: who owns each campus's booking data; where the authoritative no-overlap decision lives for each room; what a booking means while two campuses cannot reach each other; which contracts each campus offers, and who may change them. Consistency across distant sites is not solved in this lecture; the record names the questions and keeps the decision open.

Principle · Architecture decisions should explain which requirements they protect and which assumptions would change the choice.

In short · Write down what a choice protects and what would change it.

Eight principles, six checks

One principle per scene, then six checks: decide your answer before you open each one.

Principles

1 · BRIDGE

Architecture views should state their level; logical separation does not automatically imply network separation.

In short: Say which level a box lives on.

`week-06.html#/architecture-zoom`

2 · ANCHOR

Interaction structure determines which failures affect which user promises.

In short: What you wait for decides which failures reach you.

`week-06.html#/request-journey`

3 · BRIDGE

Choose deployment boundaries for demonstrated needs; modularity and distribution are different decisions.

In short: Split a process only when a real need asks.

`week-06.html#/monolith-and-services`

4 · ANCHOR

An invariant shared by concurrent operations needs enforcement that survives their interleavings.

In short: A shared rule needs one guard that races cannot slip past.

`week-06.html#/concurrent-booking`

5 · ANCHOR

Retries need defined semantics for identity, repeated effects, and uncertain outcomes.

In short: Give each operation an identity, so a retry cannot double it.

`week-06.html#/retry-and-idempotency`

6 · BRIDGE

Asynchronous work changes the timing and failure contract; it does not remove responsibility for completion.

In short: Later work is still your work: track it until done.

`week-06.html#/async-and-consistency`

7 · BRIDGE

Compatibility is a relationship between actual producers and consumers, not a label on a release.

In short: Ask who reads a field before you change it.

`week-06.html#/contracts-and-evolution`

8 · BRIDGE

Architecture decisions should explain which requirements they protect and which assumptions would change the choice.

In short: Write down what a choice protects and what would change it.

`week-06.html#/architecture-decision`

Checks: decide your answer before reading the one below each question

1. Why can two availability checks both return true?

> They can observe the same earlier state before either write has been enforced.

2. Why not block confirmation on every notification attempt?

> It couples the booking promise to an external dependency unnecessarily under R-04; separate work still needs reliable tracking.

3. What should happen when an operation key is reused with a different payload?

> A defined conflict or rejection, not silently treating the altered request as the original operation.

4. Is a C4 container always a Docker container?

> No. It denotes an application or data-store boundary in that model, not a specific packaging technology.

5. What survives a crash after an atomic booking-and-pending-notification commit?

> The booking and the pending work remain recorded; delivery still needs recovery, and a later retry can repeat a delivery attempt.

6. A batch importer crashes after storing a record but before marking its input complete. What makes a restart safe?

> Define a stable identity for the intended operation and coordinate its effect with durable completion evidence, or make replay preserve the intended result. A crash does not establish that no write occurred; reusing an identity for changed input needs an explicit policy.

What carries forward, and where to read more

Next week, several people change this system together. The contracts and the decision record travel with them.

Week 7 starts from this question:

How can several people change one system without losing intent or trust?

Carried forward to Week 7

- The booking contract: request, response, error meanings and the operation-key retry rule.
- DR-002, one modular application, an authoritative booking store, recoverable notification work, with its revisit triggers.
- One atomic decision per room protects R-01 when requests race.

In practice · examples from 2026

HTTP contracts are often described in OpenAPI; PostgreSQL can enforce a no-overlap rule for time ranges with an exclusion constraint; queued work commonly runs on RabbitMQ or Kafka; and services ship as Docker images, which are a packaging format, not the containers of the C4 model.

Sources for this lecture

S01 IEEE Computer Society, Guide to the Software Engineering Body of Knowledge (SWEBOK V4.0a) https://www.computer.org/education/bodies-of-knowledge/software-engineering

S02 ACM / IEEE-CS / AAAI, Computer Science Curricula 2023, final report https://csed.acm.org/final-report/

S08 Simon Brown, the C4 model https://c4model.com/

S09 IETF, RFC 9110: HTTP Semantics https://www.rfc-editor.org/rfc/rfc9110.html

S10 PostgreSQL documentation, Transaction Isolation https://www.postgresql.org/docs/current/transaction-iso.html

S52 OpenAPI Initiative, OpenAPI Specification 3.2.1 https://spec.openapis.org/oas/v3.2.1.html

Terms used this week

Architecture view
A drawing of a system at one stated level of detail.

System context
The whole system, its users and the outside systems it depends on.

Container (C4)
A separately running application or data store; not necessarily a Docker container.

Component
A part with one responsibility inside one running application.

Deployment boundary
Where one separately released and run piece ends and another begins.

Synchronous call
The caller waits for the answer before it continues.

Asynchronous work
Work recorded now and completed later, while the caller moves on.

Timeout
The waiting side stops waiting; it says nothing about the other side.

Unknown outcome
The requester cannot tell whether the operation took effect.

Invariant
A rule that must hold in every state, such as R-01.

Interleaving
The order in which the steps of concurrent operations actually run.

Atomic
Done as one indivisible step: nothing happens in between.

Isolation level
How much a database transaction can see of others that run at the same time.

Idempotent
Repeating the operation has the same effect as doing it once.

Operation key
An identity for one intended action, so a retry can be recognized.

Transactional outbox
Pending work committed together with the change that caused it.

Compatibility
Whether a producer and its actual consumers still work together after a change.